

TokenSage

Token Optimization for AI Coding Tools

Reduces LLM token usage by 50–95% while preserving full coding accuracy

MCP Server · Claude Code · Cursor · Cline · Codex CLI

June 2026

The Problem

When an AI reads your codebase it reads everything — full source files, raw logs, entire conversations. A single large TypeScript file can consume 8,000–15,000 tokens. A noisy build log can consume 5,000. Multiply that across a debugging session and you burn through your context window fast, hit rate limits, and pay high API costs.

TokenSage solves this by giving the model a compressed structural view instead of raw content — processing everything locally with zero external API calls.

How It Works

Architecture

TokenSage is an MCP (Model Context Protocol) server that speaks over stdio transport. When Claude or Cursor calls a tool, the server processes files locally and returns a compressed summary. The LLM never sees the full source.

```
AI Tool (Claude / Cursor / Cline)
%   calls MCP tool
%/%

MCP Server  !•!'  Code Parser + Compressor
%           Log Compressor
%           Semantic Scorer
%/%

Dashboard  http://localhost:7450  (live analytics)
```

Token Counting

Uses gpt-tokenizer with cl100k_base encoding — same as GPT-4 and Claude. Accurate to ±1% on typical code and English. Every tool call records: original tokens, optimized tokens, saved tokens, and percentage saved. All tracked in an in-memory SessionTracker singleton.

8 Compression Tools

TOOL	SAVINGS	WHAT IT DOES
<code>compress_file</code>	60–90%	Source !' purpose + symbols + imports + exports. A 5,000-line Express server (~10k tokens) !' 20-symbol summary (~400 tokens).
<code>compress_directory</code>	70–95%	Repo !' architecture + dependency graph + important files. Detects tech stack, entry points, and architectural patterns.
<code>summarize_logs</code>	80–95%	Raw logs !' status + unique errors + recommended actions. Strips timestamps, deduplicates errors, collapses stack traces.
<code>summarize_conversation</code>	60–85%	Long chat !' goals + tasks + decisions + blockers. Optionally preserves last N messages verbatim.
<code>detect_duplicates</code>	50–90%	Removes repeated stack traces via exact hash match + Jaccard similarity (~80% similar = same error).

semantic_relevance

70-95%

context_budget	Planning	Ranks files by query relevance. "Which files relate to auth?" — loads only those, skips everything else.
token_usage_report	Analytics	Token cost estimator. Fits items within a budget using priority tiers (critical/high/medium/low). Session + all-time savings. Per-tool breakdown. Resets on demand.

Real Savings Example

Debugging a TypeScript repo — without TokenSage

STEP	TOKENS
Read src/server/index.ts (full file)	8,000
Read src/middleware/auth.ts (full file)	5,000
Describe dependency graph manually	2,000
TOTAL	15,000

Same task — with TokenSage

STEP	TOKENS	SAVED
compress_file on index.ts	400	95%
compress_directory on src/	600	88%
semantic_relevance !' auth error	200	90%
TOTAL	1,200	92%

Code Parser Internals

src/parsers/code-parser.ts — regex-based, no native dependencies, cross-platform. Supports: TypeScript, JavaScript, Python, Go, Rust, Java, Ruby, PHP, Kotlin, Swift, C/C++.

For each file: (1) detect language from extension !' (2) extract imports, exports, functions, classes, interfaces, types, enums with signatures !' (3) infer file purpose from filename patterns !' (4) deduplicate imports. No AST libraries. Works on partial or broken code.

Live Dashboard

Fastify HTTP server at localhost:7450 (hash-derived port 7100–7999 per project). Shows: session token savings + percentage, per-tool call

counts and savings, token flow visualization (original !' optimized !' saved), active projects and sessions, daemon status.

Auto-Compression Hooks (Claude Code)

Beyond MCP tools, TokenSage installs Claude Code hooks that fire automatically on every interaction:

pre-read hook	Intercepts file reads. Compresses code before Claude sees it. Only fires on files >threshold.
user-prompt hook	Compresses large code/log blocks embedded in user prompts (fenced ``` blocks >30 lines, >20% savings).
post-tool-edit hook	Tracks token delta for Edit operations (old_string vs new_string). Posts to dashboard.
post-bash hook	Tracks bash command output tokens.
session-start hook	Activates caveman mode and displays session context banner.

Installation

Claude Code / Cursor / Cline

```
{
  "mcpServers": {
    "token-sage": {
      "command": "node",
      "args": ["/path/to/token-sage/dist/server/index.js"]
    }
  }
}
```

Build from Source

```
npm install && npm run build && npm start
```

Auto-install Claude Code hooks

```
node dist/cli/index.js install
```

Environment Variables

VARIABLE	DEFAULT	PURPOSE
DASHBOARD_PORT	hash of project path	Web UI port
DASHBOARD_HOST	localhost	Bind address
DASHBOARD_ENABLED	true	Enable/disable dashboard
MAX_FILE_SIZE_BYTES	500,000	Skip files larger than this
TOKENSAGE_NO_COMPRESS	—	Bypass all compression
TOKENSAGE_PROMPT_MIN_TOKENS	300	Prompt compression threshold
TOKENSAGE_MIN_SAVINGS_PCT	20	Min savings % to compress

Savings Summary

TOOL	TYPICAL SAVINGS	BEST FOR
compress_file	60-90%	Understanding single file structure
compress_directory	70-95%	Repo-wide architecture exploration

summarize_logs

80–95%

summarize_conversation	60–85%
detect_duplicates	50–90%
semantic_relevance	70–95%
context_budget	Planning
token_usage_report	Analytics

- Noisy build/test/docker output
 - Long chat history compression
 - Repeated errors and stack traces
 - Loading only relevant files
 - Token planning before long tasks
 - Session and cumulative stats
- Typical session savings: 70–92%.